

BUILD-SUPABASE.md

Partie C.6 — Relations entre les tables

Version : V1

Statut : Référence officielle des relations PostgreSQL dans Vitala

1. Objectif

Cette section définit les règles officielles de conception des relations entre les tables PostgreSQL de Vitala.

Elle garantit :

- l'intégrité référentielle ;
- une architecture cohérente ;
- des performances maîtrisées ;
- le respect du Domain Model.

Toutes les relations entre tables doivent respecter les conventions définies dans cette section.

2. Principes directeurs

Les relations reposent sur les principes suivants :

- Domain First
- Intégrité référentielle
- Faible couplage
- Cohérence métier
- Simplicité
- Évolutivité

Chaque relation doit avoir une justification métier clairement identifiée.

3. Relation One-to-One (1:1)

Une relation un-à-un est utilisée lorsqu'un enregistrement ne peut être associé qu'à un seul autre enregistrement.

Exemples :

- un utilisateur ↔ une identité numérique principale (UDI) ;
- un média ↔ ses métadonnées spécifiques.

Une contrainte `UNIQUE` doit garantir cette relation.

4. Relation One-to-Many (1:N)

Une relation un-à-plusieurs est utilisée lorsqu'un enregistrement peut être associé à plusieurs enregistrements enfants.

Exemples :

- une mission → plusieurs étapes ;
- un utilisateur → plusieurs notifications ;
- un Flash → plusieurs pièces jointes.

Les clés étrangères doivent assurer l'intégrité de la relation.

5. Relation Many-to-Many (N:N)

Une relation plusieurs-à-plusieurs doit toujours être modélisée au moyen d'une table de liaison.

La table de liaison peut contenir :

- les deux clés étrangères ;
- des métadonnées ;
- des dates ;
- un statut ;
- des informations d'audit.

Les relations N:N directes sont interdites.

6. Tables de liaison

Les tables de liaison doivent :

- posséder leur propre clé primaire (`UUID`) ;
- inclure les clés étrangères nécessaires ;
- respecter les conventions générales de création des tables ;
- être documentées.

Elles sont considérées comme des tables métier à part entière lorsqu'elles portent des informations supplémentaires.

7. Relations inter-domaines

Les relations entre domaines doivent être limitées au strict nécessaire.

Chaque domaine reste propriétaire de ses propres données.

Les dépendances directes entre domaines doivent être réduites afin de préserver leur autonomie.

8. Suppressions et mises à jour

Le comportement des relations lors d'une suppression ou d'une mise à jour doit être défini explicitement.

Les stratégies autorisées sont :

- `RESTRICT`
- `NO ACTION`
- `SET NULL`
- `SET DEFAULT`
- `CASCADE` (exceptionnel)

Le choix doit être documenté dans les migrations.

9. Relations optionnelles

Une relation peut être facultative uniquement lorsqu'elle correspond à un besoin métier identifié.

Dans ce cas :

- la clé étrangère peut accepter `NULL` ;
 - le comportement attendu doit être documenté.
-

10. Relations polymorphes

Les relations polymorphes (une référence pouvant pointer vers plusieurs types d'entités) doivent être évitées.

Si elles deviennent nécessaires, elles doivent être :

- documentées ;
- justifiées ;
- validées au niveau de l'architecture.

11. Intégrité référentielle

Toute relation doit être protégée par une clé étrangère.

Aucune relation logique ne doit exister uniquement dans le code de l'application.

La base de données reste garante de la cohérence des références.

12. Performances

Les relations doivent être optimisées :

- par des index adaptés sur les clés étrangères ;
- par des jointures maîtrisées ;
- par une limitation des dépendances inutiles.

Les relations ne doivent pas entraîner de dégradation significative des performances.

13. Documentation

Chaque relation doit être documentée avec :

- les tables concernées ;
 - le type de relation ;
 - les clés utilisées ;
 - le comportement en cas de suppression ou de mise à jour ;
 - le domaine propriétaire.
-

14. Évolution

Toute modification d'une relation doit :

- être réalisée via une migration ;
 - préserver les données existantes ;
 - être précédée d'une analyse d'impact ;
 - être documentée.
-

15. Critères de conformité

Les relations sont conformes lorsque :

- elles reflètent les besoins métier ;
 - elles utilisent des clés étrangères ;
 - elles respectent les conventions Domain-Driven ;
 - elles préservent l'intégrité référentielle ;
 - elles sont documentées ;
 - elles sont optimisées pour les performances.
-

16. Conclusion

Les relations entre les tables constituent la structure fondamentale de la base de données de Vitala.

Une conception rigoureuse des relations garantit une plateforme cohérente, évolutive, performante et fidèle au Domain Model défini dans le Build Blueprint.